LAPORAN PRAKTIKUM MOBILE PROGRAMMING
MODUL X – DESAIN GUI, NAVIGASI, DAN INTEGRASI API DENGAN JSON SERIALIZATION

Nama : Erai Bagusalim

NIM : 240605110088

Kelas : Mobile Programming Practicum A

Tanggal : 1 Oktober 2026

JURUSAN TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM MALANG

GANJIL 2026/2027

I. Tujuan

1. Memahami prinsip perancangan antarmuka grafis (Graphical User Interface / GUI) pada aplikasi Flutter yang responsif, modular, dan terstruktur menggunakan kombinasi widget ListView.builder, Card, ListTile, Stack, dan SingleChildScrollView.

2. Mengimplementasikan arsitektur navigasi antarhalaman berbasis Named Route pada Flutter menggunakan MaterialApp dengan parameter routes dan initialRoute, serta mekanisme pengiriman argumen (arguments) melalui ModalRoute.

3. Memahami dan menerapkan pembuatan model data (data modeling) di Dart menggunakan pustaka json_annotation dan generator json_serializable melalui build_runner untuk proses serialisasi (toJson) dan deserialisasi (fromJson).

4. Mengintegrasikan data dari REST API eksternal (FreeToGame API) menggunakan paket http secara asinkron (asynchronous programming) dengan memanfaatkan Future dan FutureBuilder.

5. Menampilkan data game secara dinamis baik pada halaman katalog utama (Home) maupun halaman rincian spesifikasi game (Detail), serta mengoptimalkan penanganan state dan caching future untuk mencegah pemanggilan API berulang (redundant requests).

II. Langkah Kerja

A. Tahap 1 – Halaman Home (Katalog Game dan Integrasi API)

1. Inisialisasi Proyek dan Konfigurasi Dependensi (pubspec.yaml)

Langkah pertama dimulai dengan menyiapkan proyek Flutter bernama game_app (pada implementasi praktikum mahasiswa diorganisasikan dalam folder proyek h10 dan h10a). Struktur modular diterapkan pada folder lib/ dengan membaginya ke dalam tiga subdirektori utama: lib/model/ untuk kelas entitas data, lib/view/ untuk antarmuka pengguna (UI), dan lib/viewmodel/ untuk logika komunikasi jaringan REST API. Pada berkas pubspec.yaml, ditambahkan dependensi eksternal yang dibutuhkan yaitu http untuk permintaan jaringan, json_annotation untuk anotasi model data, dan readmore untuk fitur pemotongan teks deskripsi. Pada bagian dev_dependencies ditambahkan build_runner dan json_serializable untuk pembuatan kode generator otomatis. Selanjutnya perintah flutter pub get dijalankan di terminal untuk mengunduh dan menyinkronkan seluruh paket dependensi.

2. Pengaturan Berkas Utama main.dart dan Transparansi Status Bar

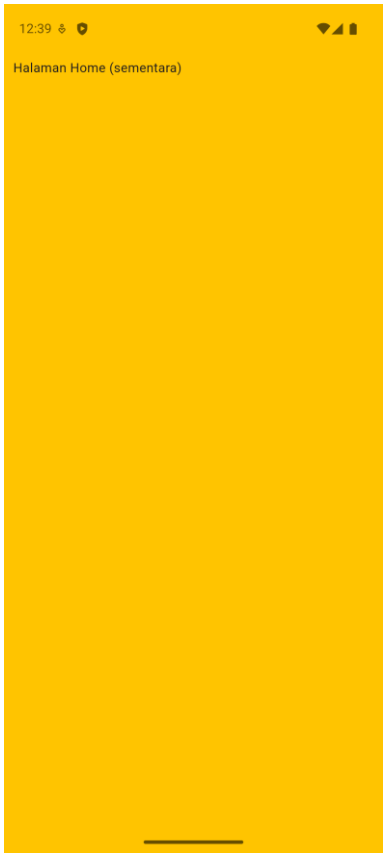
Berkas lib/main.dart dikonfigurasi sebagai titik masuk utama (entry point) aplikasi. Pustaka material, services, dan berkas view home.dart diimpor ke dalam program. Kelas MyApp dibuat sebagai StatelessWidget yang mengatur gaya tampilan bilah status (status bar) sistem operasi menggunakan SystemChrome.setSystemUIOverlayStyle agar transparan dengan ikon berwarna gelap (Brightness.dark). Metode build() mengembalikan widget MaterialApp dengan properti debugShowCheckedModeBanner: false, judul 'Flutter Game Store', dan menetapkan Home() sebagai halaman beranda awal sebelum sistem navigasi bernama (named route) diterapkan secara penuh.

3. Persiapan Berkas home.dart dan Struktur State Awal

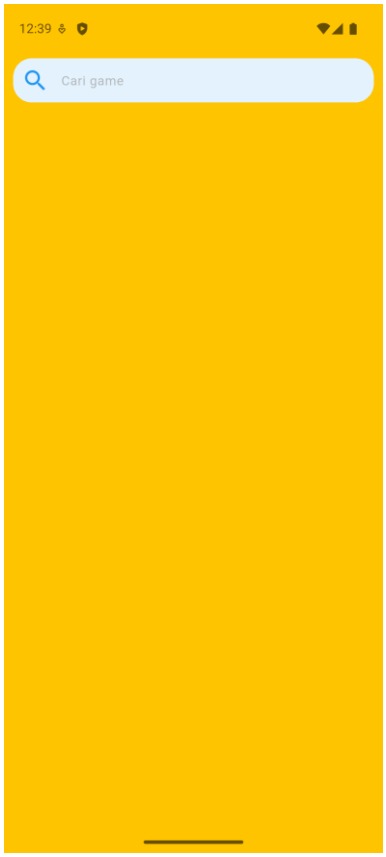
Pada folder lib/view/, dibuat berkas baru bernama home.dart yang memuat kelas Home bertipe StatefulWidget. Pada bagian awal berkas, diimpor berkas model data game.dart serta berkas logika pengambilan data fetchgame.dart. Kelas _HomeState didefinisikan sebagai turunan dari State<Home>. Variabel penampung late Future<List<Game>> gameList dideklarasikan untuk menampung objek Future berisi daftar game dari API. Pada siklus hidup initState(), fungsi loadGames() dipanggil saat widget pertama kali dimasukkan ke dalam widget tree, sehingga proses pengambilan data jaringan langsung berjalan.

4. Perancangan Kerangka Dasar Scaffold dan Penambahan Search Bar

Metode build() pada kelas _HomeState diubah untuk menyusun kerangka visual utama. Widget Scaffold digunakan dengan warna latar belakang kuning cerah (Colors.amberAccent.shade400). Bagian body dibungkus dengan SafeArea dan Container berpading horizontal 10 dan vertikal 5. Pada tahap awal, Column hanya memuat teks 'Halaman Home (sementara)'. Selanjutnya, dibuat fungsi pembantu _searchBar() yang mengembalikan widget TextField dengan latar belakang biru muda (Colors.blue.shade50), sudut membulat 20 piksel, ikon kaca pembesar (Icons.search_outlined), dan teks petunjuk 'Cari game'. Di bawah bilah pencarian, disisipkan widget SizedBox(height: 10) untuk memberi jarak vertikal sebelum daftar game dimuat.



Gambar 1. Tampilan awal Halaman Home (sementara) (rekonstruksi).

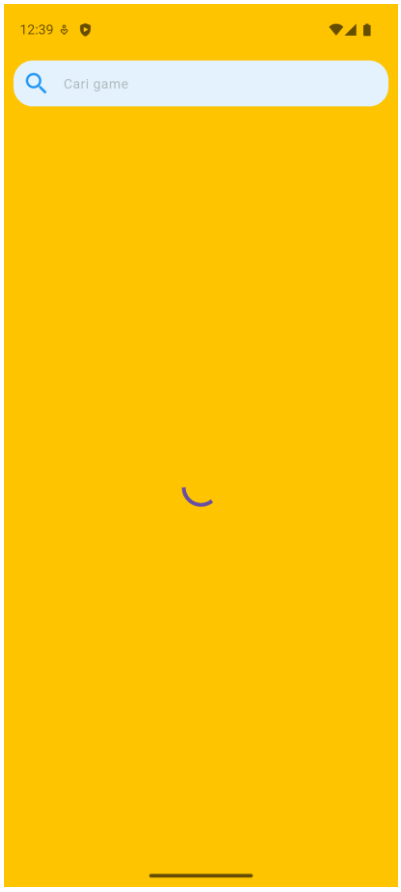


Gambar 2. Tampilan Halaman Home dengan Search Bar (rekonstruksi).

Gambar 1 memperlihatkan antarmuka awal Halaman Home saat widget Scaffold berlatar belakang amber berhasil direnderer dengan teks sementara di pojok kiri atas. Gambar 2 memperlihatkan perkembangan antarmuka setelah fungsi _searchBar() diintegrasikan ke dalam Column, menampilkan bilah pencarian rounded berikon kaca pembesar dan placeholder teks 'Cari game' yang dipisahkan oleh SizedBox.

5. Penanganan Status Pemuatan Data (FutureBuilder Loading State)

Daftar game dinamis diintegrasikan ke dalam Column menggunakan widget FutureBuilder<List<Game>> yang memanggil fetchGames() dari viewmodel. Ketika data masih dalam proses pengambilan dari server FreeToGame melalui koneksi HTTP (snapshot.connectionState == ConnectionState.waiting), antarmuka secara responsif menampilkan indikator putar CircularProgressIndicator di tengah layar agar pengguna mengetahui bahwa data sedang dimuat dari jaringan.

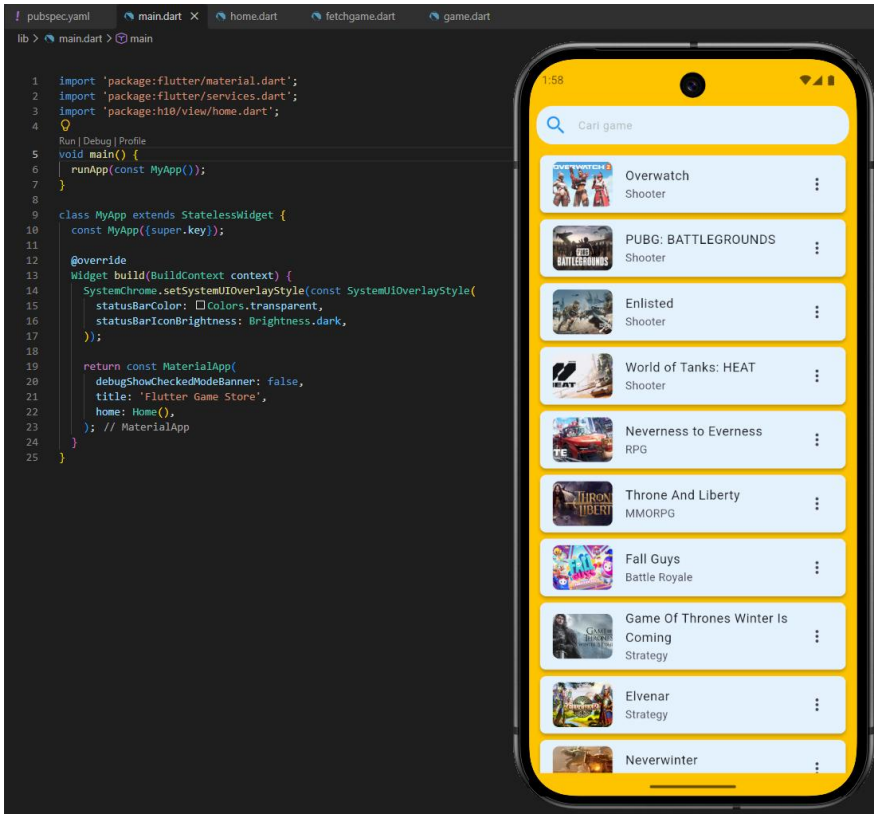


Gambar 3. Status pemuatan data (loading) dengan CircularProgressIndicator pada Halaman Home (rekonstruksi).

Tangkapan layar pada Gambar 3 memperlihatkan status pemuatan data secara asinkron pada Halaman Home. Di bawah bilah pencarian, lingkaran CircularProgressIndicator berputar di tengah layar yellow amber menandakan bahwa proses pengambilan data JSON dari endpoint FreeToGame API sedang berlangsung melalui protokol HTTP GET.

6. Pembuatan Kartu Game _listItem() dan Eksekusi Katalog Home

Untuk memformat setiap data game yang diterima dari API, dibuat fungsi Card _listItem(String urlCover, String judul, String genre) yang menghasilkan kartu berelevasi 3 dengan sudut membulat 8 piksel. Kartu memuat ListTile dengan thumbnail berdimensi 75 × 75 piksel ber-ClipRRect, judul game, genre permainan, serta trailing icon more_vert. Ketika snapshot berhasil memperoleh data, 25 entri game teratas dirender menggunakan ListView.builder di dalam Expanded. Setiap kartu game dibungkus dengan GestureDetector yang memicu perpindahan ke halaman rincian game melalui perintah Navigator.pushNamed(context, '/detail', arguments: game.id).



Gambar 4. Hasil asli Percobaan Tahap 1: Tampilan katalog game pada Halaman Home (h10).

Gambar 4 memperlihatkan hasil pengujian nyata proyek h10 pada lingkungan pengembangan Visual Studio Code (kiri) dan emulator Android (kanan). Katalog game berhasil dimuat secara penuh menampilkan kartu permainan populer seperti Overwatch, PUBG: BATTLEGROUNDS, Enlisted, World of Tanks: HEAT, Neverness to Everness, Throne And Liberty, dan Fall Guys lengkap dengan sampul gambar jaringan, judul, genre, serta bilah pencarian.

B. Tahap 2 – Halaman Detail, Navigasi Named Route, dan JSON Serialization

1. Pemodelan Data Teranotasi JSON Serialization (detailgame.dart & build_runner)

Pada folder lib/model/, dibuat berkas detailgame.dart menggunakan anotasi json_annotation. Dibuat tiga kelas model: MinimumSystemRequirements (menyimpan os, processor, memory, graphics, storage), Screenshot (menyimpan id dan url image), serta DetailGame (menyimpan metadata game lengkap). Berkas dihubungkan dengan part 'detailgame.g.dart';. Perintah terminal flutter pub run build_runner build dieksekusi untuk menghasilkan kode generator serialisasi otomatis (_$DetailGameFromJson, _$DetailGameToJson, dsb) secara akurat.

2. Pengambilan Data Rinci Game Berdasarkan ID (fetchDataFromAPI)

Pada viewmodel/fetchgame.dart, ditambahkan fungsi Future<Map<String, dynamic>> fetchDataFromAPI(int idGame) async yang mengirimkan permintaan HTTP GET ke https://www.freetogame.com/api/game?id=$idGame. Respons JSON server divalidasi dengan status kode 200 dan dikonversi menjadi objek DetailGame melalui metode factory DetailGame.fromJson(jsonData).

3. Konfigurasi Sistem Navigasi Named Route pada main.dart (/ dan /detail)

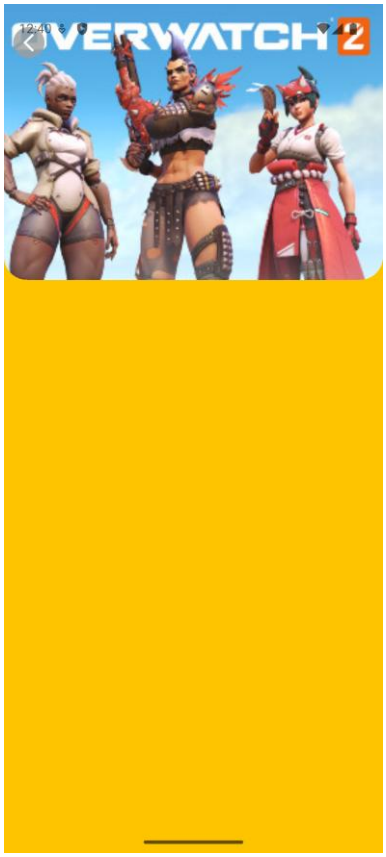
Berkas lib/main.dart diperbarui dengan menerapkan sistem navigasi Named Route terpusat. Properti home digantikan dengan initialRoute: '/' dan routes. Rute '/' memanggil const Home(), sedangkan rute '/detail' menangkap parameter ID game yang dikirim saat pengguna mengetuk item melalui final gameId = ModalRoute.of(context)!.settings.arguments as int; kemudian mengembalikan Detail(gameTerpilih: gameId).

4. Perancangan Antarmuka Halaman Detail Bertahap

Antarmuka Halaman Detail (detail.dart) dirancang melalui tahapan terstruktur: Pertama, dibuat kerangka awal Scaffold dengan AppBar bertuliskan 'Detail Game' dan teks placeholder 'Halaman Detail (sementara)'. Kedua, kerangka diganti menggunakan FutureBuilder<DetailGame> berlatar kuning amber dengan Header berupa Stack yang memuat gambar thumbnail setinggi 300 piksel bersudut bawah melengkung 25 piksel dan tombol kembali circular semi-transparan (Navigator.pop(context)).



Gambar 5. Tampilan awal Halaman Detail (sementara) (rekonstruksi).

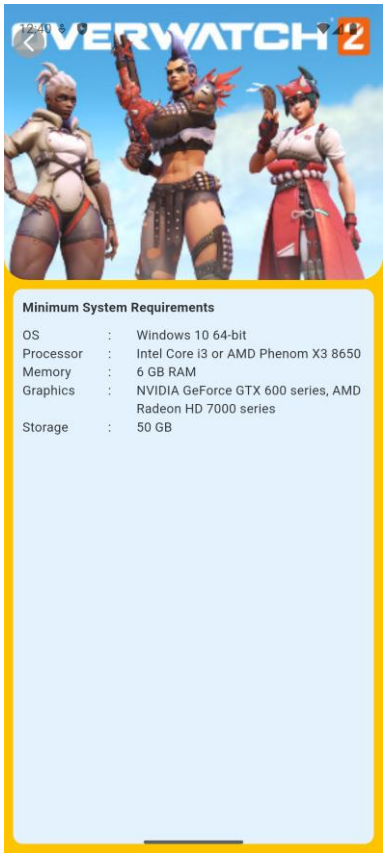


Gambar 6. Tampilan Halaman Detail dengan Header dan tombol back (rekonstruksi).

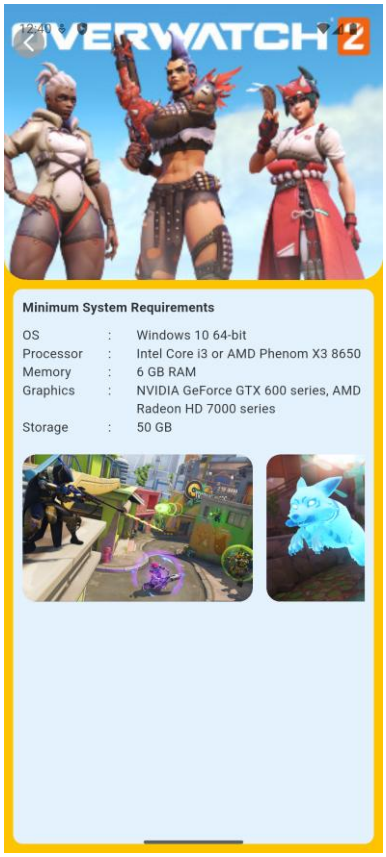
Gambar 5 memperlihatkan tampilan Halaman Detail tahap awal dengan AppBar standar dan teks placeholder di tengah layar. Gambar 6 memperlihatkan antarmuka setelah Header Stack diterapkan: foto thumbnail Overwatch 2 terpasang membentang di bagian atas dengan sudut bawah membulat dan tombol kembali melingkar putih di pojok kiri atas.

5. Integrasi Kartu Kebutuhan Sistem Minimum dan Galeri Screenshot

Di bawah Header gambar, ditambahkan widget Expanded yang membungkus Container biru muda (Colors.blue.shade50) bersudut melengkung 10 piksel. Di dalamnya, disisipkan kartu Minimum System Requirements yang menyusun baris spesifikasi OS, Processor, Memory, Graphics, dan Storage secara rapi menggunakan helper function _requirementRow(). Selanjutnya, di bawah kartu spesifikasi ditambahkan galeri tangkapan layar gameplay berupa ListView horizontal setinggi 200 piksel yang memuat gambar-gambar screenshot in-game berdimensi lebar 250 piksel dengan sudut tumpul (ClipRRect) melalui fungsi pembantu _itemList().



Gambar 7. Penambahan kartu Minimum System Requirements (rekonstruksi).

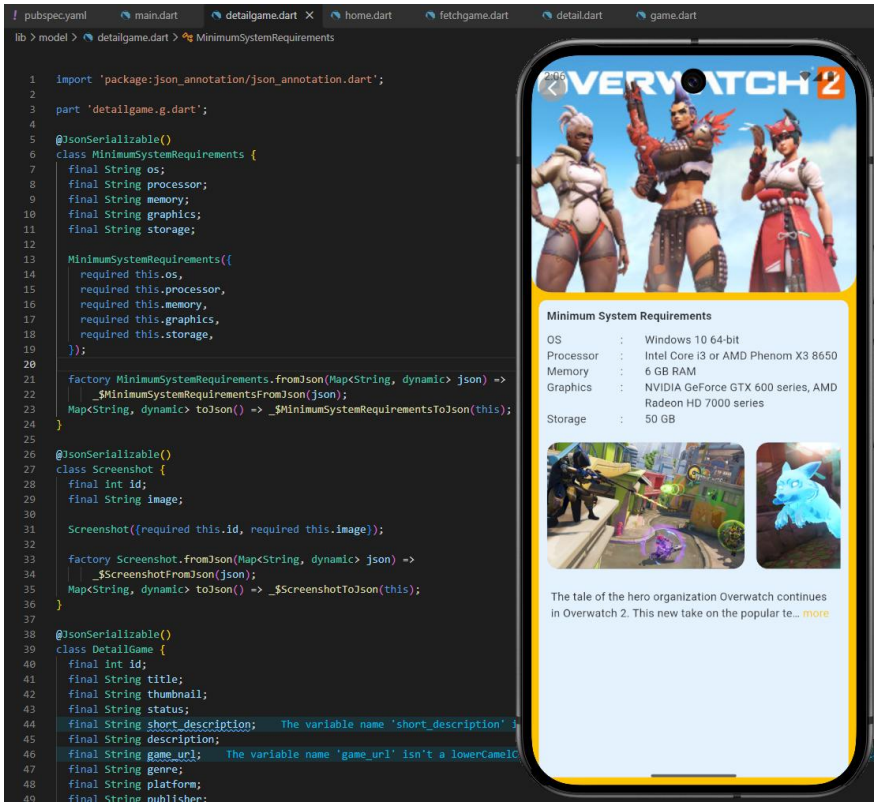


Gambar 8. Penambahan galeri screenshot horizontal (rekonstruksi).

Gambar 7 memperlihatkan kartu Minimum System Requirements berlatar biru muda yang menampilkan spesifikasi PC minimum game Overwatch 2. Gambar 8 memperlihatkan penambahan galeri screenshot permainan horizontal di bawah kartu spesifikasi sistem.

6. Penambahan Deskripsi ReadMoreText dan Eksekusi Akhir Halaman Detail

Pada bagian akhir antarmuka Detail, ditambahkan widget ReadMoreText dari paket readmore yang membatasi tampilan teks deskripsi awal menjadi 2 baris (trimLines: 2) dengan tombol interaktif 'more' dan 'less' berwarna amberAccent. Ketika pengguna mengetuk kartu Overwatch pada Halaman Home, sistem Named Route mengalirkan ID game ke rute '/detail', memicu pengambilan data rinci, dan menyajikan halaman detail lengkap yang informatif dan elegan.



Gambar 9. Hasil asli Percobaan Tahap 2: Tampilan lengkap Halaman Detail Overwatch 2 (h10).

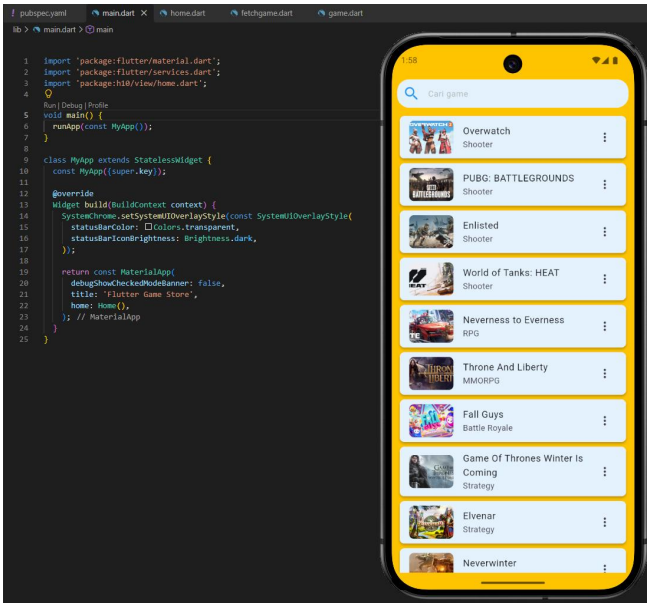
Gambar 9 menampilkan pengujian nyata Halaman Detail pada VS Code (kiri) dan emulator Android (kanan). Berkas model detailgame.dart tampak pada editor kode, sedangkan emulator menampilkan antarmuka rincian Overwatch 2 secara utuh: banner karakter Sojourn, Junker Queen, dan Kiriko, kartu spesifikasi PC minimum, karusel screenshot gameplay, serta teks sinopsis deskripsi dengan tautan 'more'.

C. Analisis Komparasi dan Optimalisasi Proyek (h10 vs h10a)

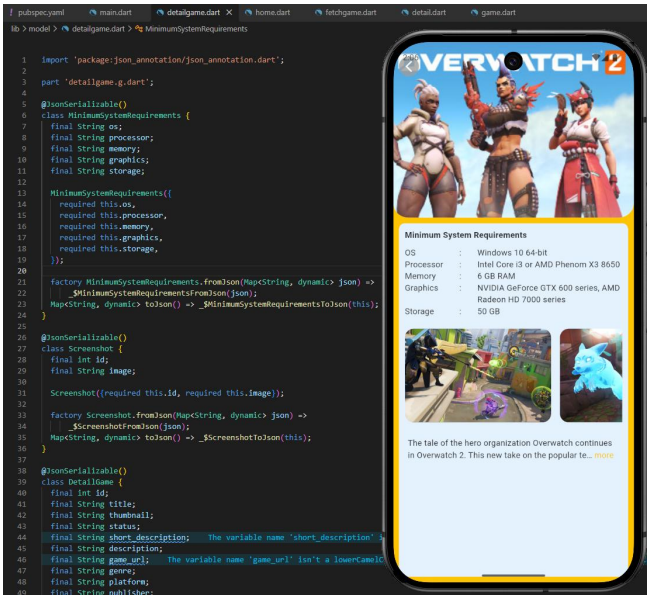
Dalam pengerjaan praktikum ini, dibangun dua versi proyek yaitu h10 dan h10a yang memiliki perbedaan struktural dan efisiensi kinerja yang fundamental: 1. Arsitektur State Caching pada FutureBuilder: Pada proyek h10, properti future pada FutureBuilder di berkas home.dart diisi langsung dengan pemanggilan fungsi future: fetchGames(). Pola ini merupakan anti-pattern di Flutter karena setiap kali widget Home mengalami rebuild (misalnya saat orientasi layar berubah, keyboard muncul, atau pengguna kembali dari halaman detail), fungsi fetchGames() akan dieksekusi kembali dari awal, memicu permintaan HTTP berulang yang memboroskan kuota internet dan menyebabkan efek kedipan (flickering). Sebaliknya, pada proyek h10a diterapkan optimalisasi arsitektural yang tepat: pemanggilan fetchGames() dilakukan satu kali

di dalam initState() dan hasilnya disimpan ke dalam variabel state gameList = fetchGames();. Properti FutureBuilder kemudian mengonsumsi future: gameList. Dengan demikian, data API di-cache dengan aman dan tidak pernah diminta ulang selama widget berada di dalam widget tree. 2. Efisiensi Formatting dan Impor: Berkas pada h10a memiliki penataan impor yang lebih rapi, penghapusan baris kode redundan, penyesuaian gaya penulisan deklarasi fungsi, serta penanganan tata letak ListView screenshot yang lebih konsisten.

III. Screenshot Hasil



Gambar 10. Screenshot Hasil Percobaan Tahap 1 – Halaman Home (Katalog Game FreeToGame).



Gambar 11. Screenshot Hasil Percobaan Tahap 2 – Halaman Detail (Informasi Game Overwatch 2).

IV. Kesimpulan

1. Desain GUI pada Flutter menawarkan fleksibilitas dan keterbacaan tinggi berkat komposisi widget modular seperti ListView.builder untuk rendering daftar data berukuran besar secara efisien, Card dan ListTile untuk pengelompokan informasi secara terstruktur, serta Stack dan SingleChildScrollView untuk penyusunan antarmuka berlapis dan responsif.

2. Sistem navigasi Named Route (/ dan /detail) memusatkan pemetaan rute aplikasi di dalam MaterialApp, mempermudah pengelolaan alur aplikasi skala besar, serta mendukung transfer parameter antarhalaman secara aman dan type-safe menggunakan ModalRoute.of(context)!.settings.arguments.

3. Penerapan JSON Serialization otomatis menggunakan paket json_annotation, json_serializable, dan build_runner terbukti menghilangkan potensi kesalahan manusia (human error) dalam memetakan respons JSON dari REST API ke dalam objek Dart, menghasilkan model data yang kuat dan mudah dirawat.

4. Integrasi REST API eksternal (FreeToGame API) menggunakan pustaka http dan widget FutureBuilder memungkinkan penanganan siklus hidup data asinkron secara komprehensif, mencakup status pemuatan (loading) dengan CircularProgressIndicator, penanganan galat jaringan (hasError), hingga penyajian data sukses.

5. Analisis komparasi antara proyek h10 dan h10a membuktikan bahwa inisialisasi pemanggilan data asinkron di dalam siklus hidup initState() yang ditampung pada variabel Future (gameList) jauh lebih optimal dibandingkan pemanggilan langsung di dalam method build(), karena mencegah request jaringan berulang dan menjaga stabilitas performa aplikasi.

Link Github :

https://github.com/Erai02/Mobile-Programming-Praktikum-A x